

Week 05: Evolutionary Computation for Engineering Design

Artificial Intelligence Applications in Engineering (MUH-920 Mühendislikte Yapay Zeka Uygulamaları). Lecture notes

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 | utkukose.com | github.com/utkukose

utkukose@sdu.edu.tr | utku.kose@und.edu | ukose@up.edu.mx | utkukose@gmail.com

Week overview, study path and tasks: <https://github.com/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-05/README.md>

Lecture page with animations and knowledge checks: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-05/lecture.html>

Interactive lab with self-assessment and reflection: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-05/lab.html>

Colab notebook with Python step 5 and the hands-on work:

https://colab.research.google.com/github/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-05/NB05_evolutionary_design.ipynb

Content licensed under CC BY 4.0. This course is updated in line with current developments in the field.

Last update: October 2026.

Overview

Design problems rarely come with gradients. The objective may be the output of a finite element model, a circuit simulator or a production line model; variables may be integers; and the landscape may have many local optima. Evolutionary algorithms search such spaces with a population of candidate designs that is improved by selection, recombination and mutation [1, 2, 3]. This week develops genetic algorithms and differential evolution, handles engineering constraints, and introduces multi-objective optimisation with Pareto fronts [4, 5, 6]. The running example is the minimum-mass design of a steel cantilever beam, whose optimum can also be found by hand, so that every algorithm can be checked against the truth.

Estimated study time: 9 to 11 hours.

Learning outcomes

By the end of the week, students are expected to formulate a design problem with variables, objective and constraints, to explain selection, crossover, mutation and elitism in a genetic algorithm, to implement a real-coded genetic algorithm and use differential evolution from SciPy, to handle constraints with penalties and feasibility rules, to interpret convergence curves over several random seeds, and to construct and read a Pareto front of two conflicting objectives.

Python in this week

These notes close with Python step 5: Random numbers, arrays and genetic operators. It covers reproducible random numbers, two-dimensional arrays, sorting and boolean selection, list

comprehensions and lambda functions, and crossover, mutation and elitism in a short genetic algorithm, applied to a population of beam designs [1, 7]. In the course materials, the step is taught at the start of the Colab notebook, where each snippet runs in a cell next to its explanation and short exercises give immediate feedback, and later sections of the notebook mark further constructs as Python moves.

Optimisation problems in engineering design

An optimisation problem has design variables, an objective to minimise or maximise, and constraints that a design must satisfy. For a beam, the variables may be the width and height of the cross-section, the objective its mass, and the constraints limits on stress and deflection. Many engineering objectives are black boxes: They are evaluated by a simulation, have no usable gradient, are noisy, or mix continuous and integer variables. Landscapes with many local optima defeat gradient methods that start from a single point.

Population-based metaheuristics address these difficulties by sampling many designs at once and moving the population towards better regions. They make few assumptions about the problem, which is their strength and their limit: The no free lunch theorems show that, averaged over all possible problems, no search algorithm outperforms any other [8]. An algorithm performs well only when its search behaviour matches the structure of the problem, so engineering knowledge about variables, scales and constraints remains decisive.

Check your understanding. Why are gradient-based methods often unsuitable for the design objectives targeted by evolutionary algorithms?

- A. Gradients are always zero in engineering problems
- B. The objective may be a black-box simulation with integer variables, noise and many local optima
- C. Evolutionary algorithms always find the global optimum
- D. Gradient methods cannot handle more than two variables

Answer: B. Black-box objectives, discrete variables and multimodality make gradients unavailable or misleading.

Formulas: An optimisation problem

An optimisation problem collects the n design variables in a vector $x = (x_1, \dots, x_n)$ and asks for the design that minimises an objective $f(x)$ subject to constraints written as $g_j(x) \leq 0$ and to bounds on every variable:

$$\min_x f(x) \text{ subject to } g_j(x) \leq 0 \text{ for } j = 1, \dots, m \text{ and } l_i \leq x_i \leq u_i$$

A design that satisfies all constraints is feasible. A maximisation problem becomes a minimisation problem by changing the sign of the objective. Writing a limit as a ratio minus one, such as $g = \sigma / \sigma_{\text{allow}} - 1$, makes all constraints dimensionless, so that $g = -0.5$ means that half of the allowance is still unused. The no free lunch theorem states the limit of every general-purpose search method [8]. If d_m is the sequence of m objective values that an algorithm a observes, then for any two algorithms a_1 and a_2 , summed over all possible objective functions f ,

$$\sum_f P(d_m | f, m, a_1) = \sum_f P(d_m | f, m, a_2)$$

Every algorithm sees the same distribution of results on average, so an advantage on one class of problems is paid for on another.

Worked example: Evaluating one beam design

The cantilever of the notebook has length $L = 2$ m and carries a tip load $P = 10$ kN. Its rectangular steel section has width b and height h , with density $\rho = 7850$ kg/m³ and Young's modulus $E = 200$ GPa. The mass is $\rho b h L$, the bending stress is $6 P L / (b h^2)$, and the tip deflection is $4 P L^3 / (E b h^3)$ [12]. The limits

are 160 MPa for the stress, $L/250=8$ mm for the deflection and $h \leq 4b$ for the slenderness. The design $b=0.05$ m and $h=0.2$ m gives:

Step 1. Mass.

$$m = 7850 \times 0.05 \times 0.2 \times 2 = 157 \text{ kg}$$

Step 2. Stress.

$$\sigma = \frac{6 \times 10^4 \times 2}{0.05 \times 0.2^2} = \frac{1.2 \times 10^5}{0.05 \times 0.04} = \frac{1.2 \times 10^5}{0.002} = 6 \times 10^7 \text{ Pa} = 60 \text{ MPa}$$

Step 3. Deflection.

$$\delta = \frac{4 \times 10^4 \times 2^3}{2 \times 10^{11} \times 0.05 \times 0.2^3} = \frac{3.2 \times 10^5}{2 \times 10^{11} \times 0.05 \times 0.008} = \frac{3.2 \times 10^5}{8 \times 10^7} = 0.004 \text{ m} = 4 \text{ mm}$$

Step 4. Constraint values.

$$g_1 = \frac{60}{160} - 1 = -0.625, g_2 = \frac{4}{8} - 1 = -0.5, g_3 = \frac{0.2}{4 \times 0.05} - 1 = 0$$

All three values are at most zero, so the design is feasible. The slenderness constraint is active, $g_3=0$, while 62.5 percent of the stress allowance and half of the deflection allowance remain unused, which suggests that a lighter feasible design exists.

Genetic algorithms

Holland described adaptation in natural and artificial systems as the evolution of a population under selection and recombination [1], and Goldberg popularised genetic algorithms as practical search and optimisation tools [2]. A genetic algorithm encodes a design as a chromosome, which may be a bit string, a vector of real numbers or a permutation. A fitness function scores each chromosome. Selection chooses parents with a preference for fitter individuals: In tournament selection, a few individuals are drawn at random and the best of them becomes a parent. Crossover combines two parents into offspring, for example by exchanging parts of their chromosomes or by blending their real values. Mutation makes small random changes that keep diversity in the population. Elitism copies the best individuals unchanged into the next generation, so that the best design found is never lost.

The balance between exploration and exploitation governs performance. Strong selection pressure and small mutations converge quickly but risk premature convergence to a local optimum; weak pressure and large mutations explore widely but converge slowly. Population size, tournament size, crossover and mutation rates are therefore design parameters of the algorithm itself, and their effect must be judged over several independent runs, because a single run of a stochastic algorithm proves little [3].

Animation: A genetic algorithm on a multimodal landscape. Each dot is a candidate design on a landscape with many local minima. Press play and watch selection concentrate the population while mutation keeps exploring. Increase the mutation strength to see exploration win over exploitation. Run it on the [lecture page](#).

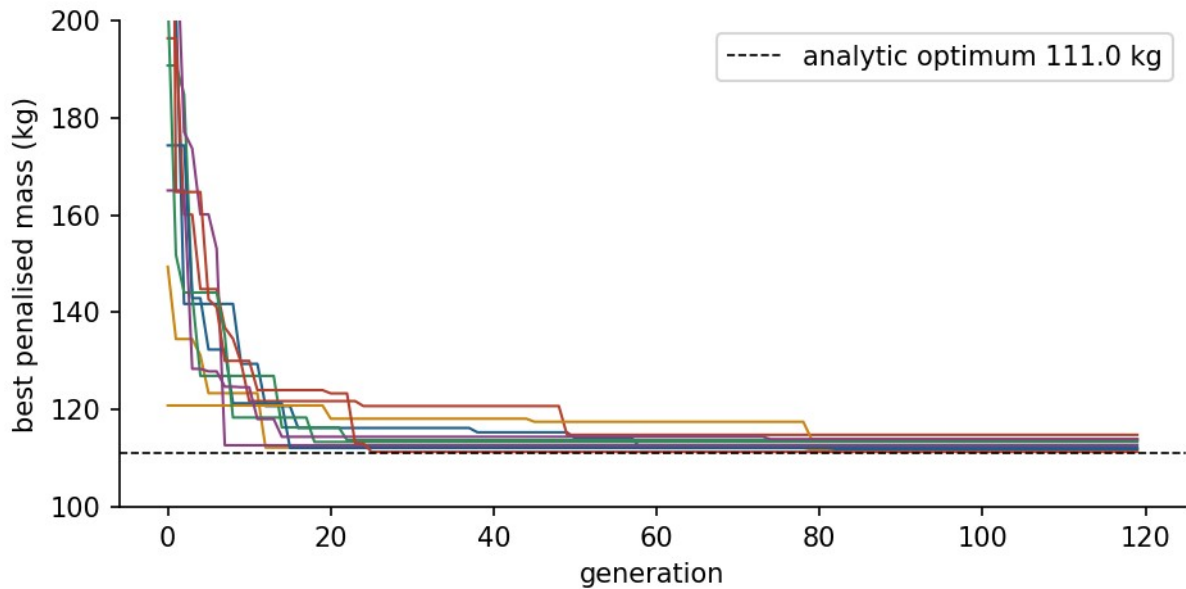


Figure 5.1. Best objective value over generations for ten independent runs of a real-coded genetic algorithm on the beam design problem. The spread between runs is part of the result.

Check your understanding. What is the role of elitism in a genetic algorithm?

- A. It increases the mutation rate for the best individuals
- B. It copies the best individuals unchanged to the next generation so the best design is never lost
- C. It removes the best individuals to preserve diversity
- D. It selects parents uniformly at random

Answer: B. Without elitism, crossover and mutation can destroy the best design found so far.

Formulas: Selection, crossover and mutation

A genetic algorithm keeps a population of N designs. For minimisation, a smaller objective value f means a fitter individual. Tournament selection draws k individuals at random, with replacement, and returns the best of them. If the population is ranked from $r = 1$ (best) to $r = N$ (worst), the individual of rank r wins when all k drawn individuals have rank r or worse but not all are worse than r :

$$P(r) = \frac{(N - r + 1)^k - (N - r)^k}{N^k}$$

Fitness-proportional or roulette-wheel selection instead picks individual i with probability $F_i / \sum_j F_j$ for positive fitness values F_i , for example $F_i = 1 / (1 + f_i)$ in a minimisation problem [2]. For real-valued chromosomes, blend crossover places a child on the line between the parents p and q , with a random weight w between 0 and 1, and Gaussian mutation adds a normally distributed random number z with mean 0 and standard deviation 1, scaled by a fraction s of the variable range $u - l$:

$$c = w p + (1 - w) q, x' = c + s(u - l)z$$

For bit-string chromosomes of length L , a string with the integer value I is decoded to the interval from l to u as

$$x = l + (u - l) \frac{I}{2^L - 1}$$

One-point crossover swaps the tails of two parents after a random cut point, and bit-flip mutation flips every bit independently with probability p_m . A string then stays unchanged with probability $(1 - p_m)^L$, and the expected number of flipped bits is $L p_m$ [1, 2]. Elitism copies the best individual unchanged into the next generation [3].

Worked example: One generation of a real-coded genetic algorithm

The objective $f(x) = (x - 3)^2 + 1$ on the interval from 0 to 7 has its minimum $f = 1$ at $x = 3$. The population holds $N = 4$ individuals, the tournament size is $k = 2$, and the mutation scale is $s = 0.05$, so that $s(u - l) = 0.05 \times 7 = 0.35$.

Step 1. Evaluation.

$$f(0.5) = (0.5 - 3)^2 + 1 = 6.25 + 1 = 7.25, f(2) = (2 - 3)^2 + 1 = 1 + 1 = 2$$

$$f(4.5) = (4.5 - 3)^2 + 1 = 2.25 + 1 = 3.25, f(6) = (6 - 3)^2 + 1 = 9 + 1 = 10$$

The ranking is $x = 2$ (rank 1), 4.5 (rank 2), 0.5 (rank 3) and 6 (rank 4), and the mean objective is $(7.25 + 2 + 3.25 + 10) / 4 = 5.625$.

Step 2. Selection pressure. With $N = 4$ and $k = 2$, so that $N^k = 16$:

$$P(1) = \frac{4^2 - 3^2}{16} = \frac{7}{16} = 0.4375, P(2) = \frac{3^2 - 2^2}{16} = \frac{5}{16} = 0.3125$$

$$P(3) = \frac{2^2 - 1^2}{16} = \frac{3}{16} = 0.1875, P(4) = \frac{1^2 - 0^2}{16} = \frac{1}{16} = 0.0625$$

The four probabilities add up to one, and the best individual is seven times as likely to become a parent as the worst.

Step 3. Elitism. The best individual, $x = 2$, enters the next generation unchanged. Three children fill the remaining places. The individuals are numbered in the order 0.5, 2, 4.5 and 6.

Child 1. The first tournament draws individuals 1 and 3, with $f = 7.25$ and $f = 3.25$, so parent $p = 4.5$ wins. The second draws 2 and 4, with $f = 2$ and $f = 10$, so parent $q = 2$ wins. With the random weight $w = 0.3$, crossover and mutation with the normal number $z = 0.4$ give

$$c = 0.3 \times 4.5 + (1 - 0.3) \times 2 = 1.35 + 1.4 = 2.75$$

$$x' = 2.75 + 0.35 \times 0.4 = 2.75 + 0.14 = 2.89, f(x') = (2.89 - 3)^2 + 1 = 1.0121$$

Child 2. The first tournament draws individuals 3 and 4, with $f = 3.25$ and $f = 10$, so parent $p = 4.5$ wins. The second draws 1 and 2, with $f = 7.25$ and $f = 2$, so parent $q = 2$ wins. With the random weight $w = 0.8$, crossover and mutation with the normal number $z = -1.2$ give

$$c = 0.8 \times 4.5 + (1 - 0.8) \times 2 = 3.6 + 0.4 = 4$$

$$x' = 4 + 0.35 \times (-1.2) = 4 + (-0.42) = 3.58, f(x') = (3.58 - 3)^2 + 1 = 1.3364$$

Child 3. The first tournament draws individuals 2 and 3, with $f = 2$ and $f = 3.25$, so parent $p = 2$ wins. The second draws 1 and 4, with $f = 7.25$ and $f = 10$, so parent $q = 0.5$ wins. With the random weight $w = 0.5$, crossover and mutation with the normal number $z = 0.9$ give

$$c = 0.5 \times 2 + (1 - 0.5) \times 0.5 = 1 + 0.25 = 1.25$$

$$x' = 1.25 + 0.35 \times 0.9 = 1.25 + 0.315 = 1.565, f(x') = (1.565 - 3)^2 + 1 = 3.059225$$

Step 4. The new generation. The population is now 2, 2.89, 3.58, 1.565, with objective values 2, 1.0121, 1.3364, 3.059225. The mean objective fell from 5.625 to

$$\frac{2+1.0121+1.3364+3.059225}{4}=1.8519$$

and the best value from 2 to 1.0121. One generation proves little, because every number above came from a random draw; the effect of the operators shows only over many generations and several independent runs [3]. Part A of the [interactive lab](#) creates such generations one at a time, with every tournament, crossover and mutation.

Worked example: Bit strings, one-point crossover and bit-flip mutation

The same interval from 0 to 7 is encoded with $L=5$ bits, so $2^5 - 1 = 31$.

Step 1. Decoding the parents. The parent 01100 has $I=0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 12$ and decodes to $x=0+7 \times 12/31=2.7097$. The parent 10011 has $I=1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 19$ and decodes to $x=7 \times 19/31=4.2903$.

Step 2. One-point crossover after the second bit. The children take the first two bits of one parent and the last three of the other: 01011 with $I=11$ and $x=7 \times 11/31=2.4839$, and 10100 with $I=20$ and $x=7 \times 20/31=4.5161$.

Step 3. Bit-flip mutation. With $p_m = 1/L = 0.2$, a child stays unchanged with probability $(1 - 0.2)^5 = 0.8^5 = 0.32768$, and on average $5 \times 0.2 = 1$ bit flips. If the last bit of 01011 flips, the child becomes 01010 with $I=10$ and $x=7 \times 10/31=2.2581$. Neighbouring integers decode to points $7/31=0.2258$ apart, which is the resolution of the encoding.

Differential evolution and other real-valued methods

Differential evolution, proposed by Storn and Price, is a simple and effective method for continuous variables [4]. For each member of the population it builds a mutant vector by adding a scaled difference of two random members to a third, $v = a + F(b - c)$, mixes the mutant with the current member by crossover with rate CR, and keeps the trial vector only if it is at least as good. Because the differences between population members shrink as the population converges, the step size adapts automatically to the scale of the landscape. SciPy provides a mature implementation with bounds and constraints [9].

Evolution strategies adapt a distribution of mutations instead. The covariance matrix adaptation evolution strategy, CMA-ES, learns the shape of promising regions and is among the most reliable methods for difficult continuous problems of moderate dimension [10]. Simulated annealing, met in Week 2, is a single-solution relative of these methods [11].

Formulas: Differential evolution and CMA-ES

Differential evolution keeps a population of N real vectors $x_1, \dots, x_N$ with D components each. For every target vector x_i it draws three other members with distinct indices r_1, r_2 and r_3 and forms the mutant vector with the scale factor F , usually between 0.4 and 1 [4]:

$$v_i = x_{r_1} + F(x_{r_2} - x_{r_3})$$

Binomial crossover builds the trial vector component by component. A random number rand_j between 0 and 1 is drawn for every component j , and one component j_{rand} is chosen at random to come from the mutant in any case:

$$u_{i,j} = v_{i,j} \text{ if } \text{rand}_j \leq \text{CR} \text{ or } j = j_{\text{rand}}, u_{i,j} = x_{i,j} \text{ otherwise}$$

On average $1 + (D - 1)$ CR components come from the mutant. Selection keeps the better of the two vectors:

$$x_i \leftarrow u_i \text{ if } f(u_i) \leq f(x_i)$$

The covariance matrix adaptation evolution strategy samples λ candidates around a mean m with a step size σ and a covariance matrix C , $x_k = m + \sigma y_k$ with y_k drawn from the normal distribution $N(0, C)$. The μ best candidates, ranked $x_{1:\lambda}, \dots, x_{\mu:\lambda}$, move the mean with positive weights w_i that add up to one, and in simplified form they also update the covariance matrix with a learning rate c_μ [10]:

$$m \leftarrow \sum_{i=1}^{\mu} w_i x_{i:\lambda}, C \leftarrow (1 - c_\mu) C + c_\mu \sum_{i=1}^{\mu} w_i y_{i:\lambda} y_{i:\lambda}^\top$$

The product $y y^\top$ of a column vector with its transpose is a matrix that stretches C along the direction of successful steps, which is how the method learns the shape of promising regions. The full algorithm adds an evolution path and a control of the step size σ .

Worked example: One trial vector of differential evolution

The objective is $f(x) = x_1^2 + x_2^2$, with its minimum 0 at the origin. The population holds four vectors, $x_1 = (1, 2)$, $x_2 = (0.5, -0.5)$, $x_3 = (-1, 1.5)$ and $x_4 = (1.5, 0.5)$, and the control parameters are $F = 0.5$ and $CR = 0.9$.

Step 1. Target and partners. The target is x_1 with $f(x_1) = 1^2 + 2^2 = 5$. The random indices are $r_1 = 2$, $r_2 = 3$ and $r_3 = 4$.

Step 2. Mutant vector.

$$x_3 - x_4 = (-1 - 1.5, 1.5 - 0.5) = (-2.5, 1), F(x_3 - x_4) = 0.5 \times (-2.5, 1) = (-1.25, 0.5)$$

$$v_1 = (0.5, -0.5) + (-1.25, 0.5) = (0.5 - 1.25, -0.5 + 0.5) = (-0.75, 0)$$

Step 3. Crossover. The index $j_{\text{rand}} = 1$ takes the first component from the mutant. For the second component $\text{rand}_2 = 0.35 \leq 0.9$, so it also comes from the mutant: $u_1 = (-0.75, 0)$.

Step 4. Selection. $f(u_1) = (-0.75)^2 + 0^2 = 0.5625 \leq 5$, so u_1 replaces x_1 . Had rand_2 been 0.95, the second component would have stayed 2, giving $u_1 = (-0.75, 2)$ with $f = 0.5625 + 4 = 4.5625$, still an improvement.

With $D = 2$, on average $1 + (2 - 1) \times 0.9 = 1.9$ components come from the mutant. The step $F(x_3 - x_4)$ is proportional to the spread of the population, so the steps shrink automatically as the population contracts around the optimum [4]. Part B of the [interactive lab](#) processes one target vector after another and shows this self-adapting step size.

Constraints

Engineering designs must be feasible. A penalty function adds to the objective a term that grows with the amount of constraint violation, which turns a constrained problem into an unconstrained one; the difficulty is choosing the weight of the penalty. Deb's feasibility rules avoid weights altogether: A feasible design always beats an infeasible one, two feasible designs are compared by objective, and two infeasible designs are compared by their total violation [5]. Repair operators that move an infeasible design back into the feasible region are useful when the constraint structure is known, and variable bounds are usually enforced directly.

For the cantilever beam of the notebook, a tip load P acts on a rectangular section of width b and height h . The maximum bending stress is $6 P L / (b h^2)$ and the tip deflection is $4 P L^3 / (E b h^3)$ [12]. Minimising mass subject to limits on stress, deflection and the ratio h / b produces an optimum on the boundary of the feasible region, where the deflection limit and the ratio limit are both active. This optimum can be derived by hand, which makes the problem a good benchmark for the algorithms.

Check your understanding. Under Deb's feasibility rules, how are two infeasible designs compared?

- A. By their objective values
- B. By their total constraint violation
- C. Randomly
- D. They are both discarded

Answer: B. Among infeasible designs, the one closer to feasibility wins, which guides the search towards the feasible region [5].

Formulas: Penalties and feasibility rules

The total violation of a design adds up the parts of the constraints that are not satisfied:

$$V(x) = \sum_{j=1}^m \max(0, g_j(x))$$

A static penalty adds the violation to the objective with a weight r , so that an unconstrained algorithm can be used:

$$\phi(x) = f(x) + r V(x)$$

Squared violations, $\max(0, g_j)^2$, are a common variant. Deb's feasibility rules compare two designs x and y without any weight [5]: If $V(x) = 0$ and $V(y) > 0$, the feasible x is better. If $V(x) = V(y) = 0$, the smaller objective is better. If both are infeasible, the smaller violation is better.

For the cantilever with $h = 4b$, the moment of inertia of the section, $I = b h^3 / 12$, enters the deflection through $b h^3 = b (4b)^3 = 64 b^4$, so the deflection limit becomes a condition on b alone:

$$\delta = \frac{4 P L^3}{64 E b^4} = \frac{P L^3}{16 E b^4} \leq \delta_{\text{allow}} \Rightarrow b \geq \left(\frac{P L^3}{16 E \delta_{\text{allow}}} \right)^{1/4}$$

The arrow reads "implies". For a given mass $\rho b h L$, that is a given area $b h$, the deflection $4 P L^3 / (E (b h) h^2)$ falls as h grows, so the lightest design pushes h to the slenderness limit and b to the deflection limit [12].

Worked example: Penalty weight against feasibility rules

Three designs of the beam:

Design	b, h (m)	Mass (kg)	Stress (MPa)	Deflection (mm)	g_1, g_2, g_3	V
A	0.05, 0.2	157	60	4	-0.625, -0.5, 0	0
B	0.04, 0.16	100.48	117.19	9.7656	-0.2676, 0.2207, 0	0.2207
C	0.03, 0.15	70.65	177.78	15.8025	0.1111, 0.9753, 0.25	1.3364

Step 1. The violations. Design A is the feasible design of the first example, $V = 0$. With $6 P L = 1.2 \times 10^5$ N m and $4 P L^3 = 3.2 \times 10^5$ N m³ as in the first example, design B gives

$$\sigma = \frac{1.2 \times 10^5}{0.04 \times 0.16^2} = \frac{1.2 \times 10^5}{0.04 \times 0.0256} = \frac{1.2 \times 10^5}{0.001024} = 117.19 \text{ MPa}$$

$$\delta = \frac{3.2 \times 10^5}{2 \times 10^{11} \times 0.04 \times 0.16^3} = \frac{3.2 \times 10^5}{2 \times 10^{11} \times 0.04 \times 0.004096} = \frac{3.2 \times 10^5}{3.2768 \times 10^7} = 9.7656 \text{ mm}$$

$$g_1 = \frac{117.19}{160} - 1 = -0.2676, g_2 = \frac{9.7656}{8} - 1 = 0.2207, g_3 = \frac{0.16}{4 \times 0.04} - 1 = 0, V = 0.2207$$

Only the deflection limit is violated. For design C:

$$\sigma = \frac{1.2 \times 10^5}{0.03 \times 0.15^2} = \frac{1.2 \times 10^5}{0.03 \times 0.0225} = \frac{1.2 \times 10^5}{0.000675} = 177.78 \text{ MPa}$$

$$\delta = \frac{3.2 \times 10^5}{2 \times 10^{11} \times 0.03 \times 0.15^3} = \frac{3.2 \times 10^5}{2 \times 10^{11} \times 0.03 \times 0.003375} = \frac{3.2 \times 10^5}{2.025 \times 10^7} = 15.8025 \text{ mm}$$

$$g_1 = \frac{177.78}{160} - 1 = 0.1111, g_2 = \frac{15.8025}{8} - 1 = 0.9753, g_3 = \frac{0.15}{4 \times 0.03} - 1 = 0.25$$

$$V = 0.1111 + 0.9753 + 0.25 = 1.3364$$

The masses are $7850 \times 0.04 \times 0.16 \times 2 = 100.48 \text{ kg}$ and $7850 \times 0.03 \times 0.15 \times 2 = 70.65 \text{ kg}$.

Step 2. A weak penalty. With $r = 100$: $\phi(A) = 157 + 100 \times 0 = 157$ and $\phi(B) = 100.48 + 100 \times 0.2207 = 122.55$. The infeasible design B wins, so the algorithm would return a beam that deflects too much.

Step 3. A strong penalty. With $r = 1000$: $\phi(B) = 100.48 + 1000 \times 0.2207 = 321.18$, now worse than A with 157. The result depends on the weight, which must be chosen for each problem.

Step 4. Feasibility rules. A beats B and C because it is feasible. Between the two infeasible designs, B beats C because $0.2207 < 1.3364$. No weight was needed.

Worked example: The lightest beam by hand

Step 1. The width.

$$b^4 = \frac{P L^3}{16 E \delta_{\text{allow}}} = \frac{10^4 \times 2^3}{16 \times 2 \times 10^{11} \times 0.008} = \frac{8 \times 10^4}{2.56 \times 10^{10}} = 3.125 \times 10^{-6} \text{ m}^4$$

$$b = (3.125 \times 10^{-6})^{1/4} = \sqrt{\sqrt{3.125 \times 10^{-6}}} = \sqrt{0.001767767} = 0.042045 \text{ m}$$

Step 2. Height and mass. $h = 4 \times 0.042045 = 0.16818 \text{ m}$, and

$$m = 7850 \times 0.042045 \times 0.16818 \times 2 = 7850 \times 0.00707113 \times 2 = 111.017 \text{ kg}$$

Step 3. The stress check.

$$\sigma = \frac{1.2 \times 10^5}{0.042045 \times 0.16818^2} = \frac{1.2 \times 10^5}{0.042045 \times 0.028285} = \frac{1.2 \times 10^5}{0.00118924} = 100.9 \text{ MPa}$$

which is below 160 MPa, so the stress constraint is inactive, as assumed. The optimum, 111.017 kg, lies on the boundary where the deflection and slenderness constraints are active, and Part C of the [interactive lab](#) uses it as the known optimum when the algorithms are compared. Part D of the [interactive lab](#) evaluates penalties and feasibility rules for any design.

Multiple objectives and Pareto optimality

Real designs trade off several objectives: mass against stiffness, cost against reliability, efficiency against noise. A design dominates another if it is no worse in every objective and better in at least one. The designs that no other design dominates form the Pareto set, and their objective values form the Pareto front. The engineer, not the algorithm, chooses a design from the front by weighing the objectives. NSGA-II finds approximations of the front in one run by ranking the population into non-dominated fronts and preserving spread with a crowding distance [6]. Simpler methods repeat single-objective optimisation, for example minimising mass for a sequence of deflection limits, which is how the notebook builds its front.

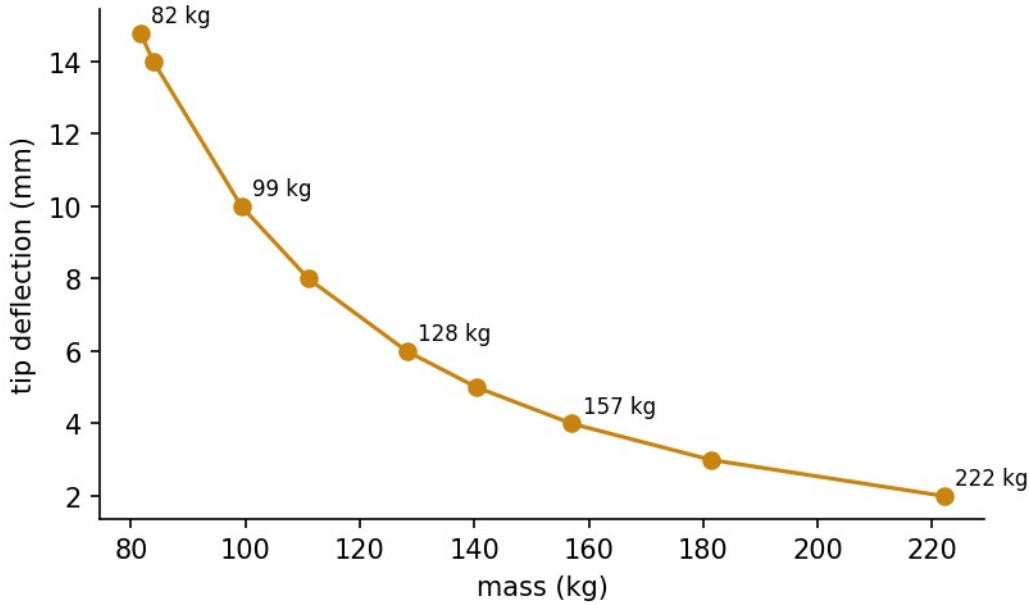


Figure 5.2. Pareto front of beam designs that trade mass against tip deflection, obtained by minimising mass for a sequence of deflection limits.

Formulas: Dominance, fronts and crowding

With objectives $f_1, \dots, f_K$ to be minimised, a design x dominates a design y , written $x < y$, when it is no worse in every objective and better in at least one:

$$x < y \Leftrightarrow f_k(x) \leq f_k(y) \text{ for all } k \text{ and } f_k(x) < f_k(y) \text{ for at least one } k$$

The double arrow reads “exactly when”. The non-dominated designs of a set form its first front; removing them and repeating the test gives the second front, and so on. NSGA-II ranks its population by these fronts and, within a front, prefers designs in sparsely populated regions. The crowding distance of a design i adds, over all objectives, the gap between its two neighbours along that objective, divided by the range of the objective in the front [6]:

$$d_i = \sum_{k=1}^K \frac{f_k(i+1) - f_k(i-1)}{f_k^{\max} - f_k^{\min}}$$

Here $i-1$ and $i+1$ are the neighbours of i when the front is sorted by objective k , and the two extreme designs receive an infinite distance so that they are always kept. The ε -constraint method of the notebook minimises one objective while it limits the other, $\min f_1(x)$ subject to $f_2(x) \leq \varepsilon$, and repeats this for a sequence of limits ε (epsilon). For the beam, the analysis of the lightest design gives every point of the front in closed form while the stress limit is inactive:

$$m(\varepsilon) = \rho L \sqrt{\frac{P L^3}{E \varepsilon}}$$

because $m = \rho L b h = 4 \rho L b^2$ and $b^2 = \sqrt{P L^3 / (16 E \varepsilon)} = \frac{1}{4} \sqrt{P L^3 / (E \varepsilon)}$.

Worked example: Fronts and crowding distances of six beam designs

Step 1. Points of the front. For a deflection limit of 12 mm:

$$m = 7850 \times 2 \times \sqrt{\frac{10^4 \times 8}{2 \times 10^{11} \times 0.012}} = 15700 \times \sqrt{3.3333 \times 10^{-5}} = 15700 \times 0.0057735 = 90.6 \text{ kg}$$

The same formula gives 157 kg for 4 mm, 128.2 kg for 6 mm and 111 kg for 8 mm. The 4 mm point is design A of the previous example, which explains its active slenderness constraint.

Step 2. Dominance. The six designs are P1 = (157 kg, 4 mm), P2 = (130 kg, 8 mm), P3 = (111 kg, 8 mm), P4 = (90.6 kg, 12 mm), P5 = (95 kg, 15 mm), P6 = (128.2 kg, 6 mm). Every pair is compared in both objectives. P3 dominates P2, because $111 < 130$ and $8 \leq 8$. P6 also dominates P2, because $128.2 < 130$ and $6 < 8$. P4 dominates P5, because $90.6 < 95$ and $12 < 15$. No other pair has a dominance relation: For example, P1 is heavier but stiffer than P6. The first front is therefore P1, P3, P4 and P6, and the second front holds P2 and P5.

Step 3. Crowding distances in the first front. Sorted by mass, the front reads P4, P3, P6, P1. The mass range is $157 - 90.6 = 66.4$ kg and the deflection range $12 - 4 = 8$ mm. P4 and P1 are extreme and receive infinite distances. For the two inner designs:

$$d_{P3} = \frac{37.6}{66.4} + \frac{6}{8} = 0.5663 + 0.75 = 1.3163$$

$$d_{P6} = \frac{46}{66.4} + \frac{4}{8} = 0.6928 + 0.5 = 1.1928$$

If NSGA-II had to drop one design of this front, it would drop P6, whose neighbours are closer together, and keep the spread of the front.

Step 4. Where the closed form ends. The stress $6 P L / (b h^2) = 6 P L / (16 b^3)$ reaches 160 MPa when $b^3 = 1.2 \times 10^5 / (16 \times 1.6 \times 10^8)$, that is at a deflection of about 14.8 mm. For larger limits the stress constraint becomes active and the front no longer follows $m(\varepsilon)$. Part E of the [interactive lab](#) ranks any set of designs and computes their crowding distances, and Part F samples beam designs and draws the Pareto front of mass and deflection.

Evolution at work

Evolutionary computation has produced designs that engineers would hardly have proposed by hand. An X-band antenna evolved at NASA for the Space Technology 5 mission met demanding requirements and flew in space in 2006 [13]. Structural shapes, truss topologies, electrical machines, chemical process conditions and production schedules are routinely optimised with genetic algorithms, differential evolution and their relatives [3]. Week 6 continues with swarm intelligence, which replaces biological evolution by the collective behaviour of birds, ants and bees.

Check your understanding. Design A has mass 100 kg and deflection 6 mm. Design B has mass 110 kg and deflection 6 mm. Which statement holds?

- A. B dominates A
- B. A dominates B

- C. Neither dominates the other
- D. Both are infeasible

Answer: B. A is better in mass and equal in deflection, so it dominates B.

Python step 5: Random numbers, arrays and genetic operators

Reproducible randomness

Evolutionary algorithms draw random numbers at every step, so repeated runs give different results. A random number generator created with `np.random.default_rng(seed)` produces the same sequence every time it starts from the same seed. Fixed seeds make experiments repeatable, and comparing algorithms over several seeds shows how much of a difference is due to chance.

```
import numpy as np

gen = np.random.default_rng(42)
print(gen.uniform(0.1, 0.5, size=3))      # three numbers between 0.1
and 0.5
print(gen.integers(1, 7, size=5))          # five dice rolls from 1 to 6
again = np.random.default_rng(42)
print(again.uniform(0.1, 0.5, size=3))    # the same three numbers as
the first line
```

Output

```
[0.40958242 0.27555138 0.44343917]
[1 5 2 1 4]
[0.40958242 0.27555138 0.44343917]
```

A population as a two-dimensional array

A population of candidate designs fits in a two-dimensional array: Each row is one design, and each column is one design variable. For a rectangular steel beam, the columns hold the width b and the height h of the section. `pop[:, 0]` selects the first column of all rows, and formulas written with arrays evaluate the whole population at once. The beam below spans 5 m and carries a bending moment of 25 kN m, and the bending stress of a rectangular section is $6 M / (b h^2)$.

```
L_m, M_Nm, rho, allow_Pa = 5.0, 25e3, 7850.0, 150e6
pop = gen.uniform([0.02, 0.05], [0.20, 0.40], size=(8, 2))  # columns:
b and h in metres
b, h = pop[:, 0], pop[:, 1]
mass = rho * b * h * L_m                                     # kg
stress = 6 * M_Nm / (b * h ** 2)                             # Pa
feasible = stress <= allow_Pa
print(pop.shape, feasible)
print(np.round(mass, 1))
```

Output

```
(8, 2) [ True  True  True  True  True False  True  True]
[2003.6  350.9 1274.6 1802.7  507.5  340.2 1797.9 1069. ]
```

25e3 is scientific notation for 25000. The bounds are given per column, so the generator draws widths between 0.02 and 0.20 m and heights between 0.05 and 0.40 m.

Sorting and selection

`np.argsort` returns the positions that would sort an array from the smallest to the largest value. Indexing an array with these positions reorders its rows, and slicing keeps the best few. Infeasible designs receive a penalty that grows with the violation of the stress limit, a common way to handle constraints in evolutionary search.

```
violation = np.maximum(stress / allow_Pa - 1, 0)      # 0 for feasible designs
fitness = mass * (1 + 10 * violation)                # penalised mass
order = np.argsort(fitness)                          # positions from best to worst
best3 = pop[order[:3]]
print(order)
print(np.round(best3, 3))
```

Output

```
[1 4 7 2 6 3 0 5]
[[0.043 0.208]
 [0.1   0.13 ]
 [0.156 0.174]]
```

Comprehensions and lambda functions

A list comprehension builds a list in one line, in the form `[expression for item in sequence if condition]`, and replaces a loop that appends to a list. A lambda is a short function without a name, often used as a sorting key. `min` with the `key` argument returns the item whose key is smallest.

```
labels = [f"b={bb:.2f} h={hh:.2f}" for bb, hh in pop[:3]]
print(labels)
light = [i for i in range(len(pop)) if mass[i] < 500]
print("designs lighter than 500 kg:", light)
area = lambda bb, hh: bb * hh
print(np.round(min(pop, key=lambda d: area(d[0], d[1])), 3))
```

Output

```
['b=0.16 h=0.33', 'b=0.04 h=0.21', 'b=0.09 h=0.37']
designs lighter than 500 kg: [1, 5]
[0.12  0.072]
```

Crossover, mutation and one generation

Selection alone only reorders existing designs. Genetic algorithms create new ones with two operators [1]. Crossover mixes two parents: For real-valued variables, arithmetic crossover takes a weighted mean, $\text{child} = \alpha \text{parent1} + (1 - \alpha) \text{parent2}$, with a random weight α between 0 and 1. Mutation adds a small random change to each variable, and `np.clip` pulls values that leave the allowed range back to the nearest bound.

```
low, high = np.array([0.02, 0.05]), np.array([0.20, 0.40])
p1, p2 = pop[order[0]], pop[order[1]]                # the two best designs as parents
alpha = gen.uniform(0, 1)
child = alpha * p1 + (1 - alpha) * p2                # arithmetic crossover
child = child + gen.normal(0, [0.005, 0.01])          # Gaussian mutation, one scale per variable
child = np.clip(child, low, high)                     # stay inside
```

the bounds

```
print(np.round(p1, 3), np.round(p2, 3), "->", np.round(child, 3))
```

Output

```
[0.043 0.208] [0.1 0.13] -> [0.044 0.201]
```

A function that returns the penalised mass of a whole population lets the operators run in a loop. The loop below evolves 20 designs for 60 generations. The best five designs survive unchanged, which is called elitism and guarantees that the best design found so far is never lost, and fifteen children of randomly chosen parents from the better half fill the rest of the population. `np.vstack` stacks the survivors and the children into the next population.

```
def penalised_mass(P):
    """Penalised mass in kg of every design (row) of P."""
    m = rho * P[:, 0] * P[:, 1] * L_m
    s = 6 * M_Nm / (P[:, 0] * P[:, 1] ** 2)
    return m * (1 + 10 * np.maximum(s / allow_Pa - 1, 0))

evo = np.random.default_rng(1)
P = evo.uniform(low, high, size=(20, 2))
for g in range(60):
    P = P[np.argsort(penalised_mass(P))] # best design
    first
    kids = []
    for _ in range(15):
        a, b_ = P[evo.choice(10, size=2, replace=False)]
        w = evo.uniform(0, 1)
        kids.append(np.clip(w * a + (1 - w) * b_ + evo.normal(0,
[0.002, 0.005]), low, high))
    P = np.vstack([P[:5], kids]) # elitism: the
best five survive unchanged
    if g % 20 == 0:
        print(f"generation {g:2d}: best penalised mass
{penalised_mass(P).min():.1f} kg")
    best = P[np.argmin(penalised_mass(P))]
    print(f"best design: b = {best[0]:.3f} m, h = {best[1]:.3f} m, mass
{penalised_mass(best[None, :])[0]:.1f} kg")
```

Output

```
generation 0: best penalised mass 250.2 kg
generation 20: best penalised mass 175.6 kg
generation 40: best penalised mass 175.6 kg
best design: b = 0.020 m, h = 0.224 m, mass 175.6 kg
```

A hand calculation gives the lightest feasible section of this problem: The width sits at its lower bound of 0.02 m, and the height just satisfies the stress limit, $h = 0.224$ m, for a mass of 175.5 kg. The genetic algorithm approaches this design from random starting points without any derivative, which is what makes it useful when no formula for the optimum exists.

Check your understanding. What does `[x ** 2 for x in range(4) if x % 2 == 0]` produce?

- A. [0, 4]
- B. [0, 1, 4, 9]
- C. [4]
- D. [1, 9]

Answer: A. range(4) yields 0, 1, 2 and 3. Only 0 and 2 are even, and their squares are 0 and 4.

Discipline challenges

Evolutionary algorithms need only a way to evaluate a design. Choose a row, write the objective and constraints as a Python function, and let the algorithms search.

Department	Challenge
Civil Engineering	Minimum-cost reinforced concrete beam section with strength and serviceability constraints.
Mechanical Engineering	Spring or shaft design with stress, deflection and fatigue constraints.
Electrical and Electronics Engineering	Placement of capacitor banks in a distribution feeder to reduce losses.
Industrial Engineering	Job sequencing on a machine to minimise total tardiness, with a permutation chromosome.
Chemical Engineering	Operating temperature and residence time of a reactor that maximise yield under a safety limit.
Food Engineering	Blend of ingredients that meets nutritional targets at minimum cost.
Environmental Engineering	Location and capacity of monitoring stations or treatment units under a budget.
Mining Engineering	Blast pattern parameters that balance fragmentation and vibration.
Textile Engineering	Yarn blend proportions for target strength and cost.
Automotive Engineering	Gear ratios that trade acceleration against fuel consumption, as a two-objective problem.
Physics and Mathematics	Fitting parameters of a nonlinear model to data with a multimodal error surface, compared with gradient methods.

Weekly task

Formulate a design problem of your department with at least two variables and two constraints, solve it with the genetic algorithm of the notebook and with SciPy's differential evolution over at least five seeds each, and report the best, median and worst results with convergence curves. If the problem has two natural objectives, construct a Pareto front. Discuss the results in about 500 words and cite at least three works from this week's references [2, 4, 5].

Research and report assignment (optional)

Evolutionary design in engineering. Review evolutionary or other population-based optimisation in one engineering field, such as structural optimisation, antenna and circuit design, process optimisation or scheduling. Discuss how constraints and multiple objectives were handled, how results were validated, and how the no free lunch theorems apply to the claims made [6, 8, 13].

Both assignments are optional. During an active semester, the weekly task can be sent together with the learning log exported from the interactive lab, and the research report on its own, to utkukose@sdu.edu.tr or utkukose@gmail.com. They carry no separate weight, but the instructor may take them into account as a discretionary adjustment of the midterm component of the grade. The assessment section of the syllabus gives the details, and research reports follow the report template of the course repository.

References

- [1] Holland, J. H. (1992). *Adaptation in Natural and Artificial Systems* (2nd ed.). MIT Press. First edition 1975, University of Michigan Press.
- [2] Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
- [3] Eiben, A. E., & Smith, J. E. (2015). *Introduction to Evolutionary Computing* (2nd ed.). Springer. <https://doi.org/10.1007/978-3-662-44874-8>
- [4] Storn, R., & Price, K. (1997). Differential evolution: A simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4), 341-359. <https://doi.org/10.1023/A:1008202821328>
- [5] Deb, K. (2000). An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, 186(2-4), 311-338. [https://doi.org/10.1016/S0045-7825\(99\)00389-8](https://doi.org/10.1016/S0045-7825(99)00389-8)
- [6] Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182-197. <https://doi.org/10.1109/4235.996017>
- [7] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- [8] Wolpert, D. H., & Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), 67-82. <https://doi.org/10.1109/4235.585893>
- [9] Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261-272. <https://doi.org/10.1038/s41592-019-0686-2>
- [10] Hansen, N., & Ostermeier, A. (2001). Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2), 159-195. <https://doi.org/10.1162/106365601750190398>
- [11] Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671-680. <https://doi.org/10.1126/science.220.4598.671>
- [12] Gere, J. M., & Goodno, B. J. (2013). *Mechanics of Materials* (8th ed.). Cengage Learning.
- [13] Hornby, G. S., Lohn, J. D., & Linden, D. S. (2011). Computer-automated evolution of an X-band antenna for NASA's Space Technology 5 mission. *Evolutionary Computation*, 19(1), 1-23. https://doi.org/10.1162/EVCO_a_00005